

1.2 The Art of Prompting

The input or question in AI chat systems is usually referred to as a *prompt*. Asking the right questions has a major influence on how good or effective the results are. The art of optimally formulating the prompt is often referred to as *prompt engineering*. This section summarizes some tips on this topic:

- **Additional information**

Introduce the question with contextual information about the programming language, library, tool, or environment you're using.

If you ask for a regular pattern in general, ChatGPT has to guess which syntax variant is meant (Portable Operating System Interface [POSIX], POSIX Extended Regular Expressions [ERE], Perl Compatible Regular Expressions [PCRE], etc.). However, if you specify that you want to use the regular pattern in the C# language, ChatGPT can take the Microsoft-specific features of regular expressions into account.

It's sufficient to briefly prefix the first prompt of a session with the programming language or another specification, such as "bash/Linux", "nodejs 20", "PHP 8", "Regex/PCRE", "SQL Server", or "zsh/macOS".

- **Additional conditions**

Indicate what is important to you:

- Do you want short, compact code?
- Should as few imports/modules/libraries as possible be used?
- Is the code security-critical?
- Is maximum efficiency in execution crucial?
- Should the code be formulated recursively, if possible? Or, on the contrary, should recursion be avoided?

You can always dispense with such additional rules in the first attempt and wait and see what the result looks like. Afterward, you can still ask the AI tool to reformulate the code in compliance with new conditions:

The code works, but is too slow. Is it possible to make it faster?

- **Formal rules**

You can define formal rules. These can relate to the maximum line length, the desired convention for variable and function names (camel case, snake case, etc.), the location and extent of comments, and so on.

- **Language**

Even if English is not your mother language, try to formulate your question in English.

- **Follow-up questions**

Ask follow-up questions. ChatGPT automatically takes the question/answer history into account.

- **A second try never hurts**

If you're not satisfied with an answer, just try again. Clearly state which detail of the answer doesn't fit from your point of view.

In the ChatGPT web interface, you can click on the **Regenerate** button.

The new answer happens to be a little different and may be more in line with your expectations. But as long as ChatGPT doesn't know why you're dissatisfied, the chances of a better result on the second attempt are relatively low. It's better if you state in a follow-up question what it was you didn't like in the previous answer.

If there is no **Regenerate** button and you don't have the time or desire to formulate specific improvement requests, you can try the following universal instruction: Do better! Don't worry about the simplistic nature of this prompt; it requires little effort and occasionally even leads to success.

- **Multiple options**

Usually, you ask a question and get *one* answer, but it may not be the best answer from your particular perspective.

If you're familiarizing yourself with a new area or the solution is unclear, you should also express this in the prompt. Ask the language model to give you several options to choose from, for example:

What options do I have to implement a REST API in Python?

The AI tool will present an overview of various libraries and frameworks. You can research details of the proposals on the internet or in the AI chat system. Then, decide on one of the variants and ask follow-up questions.

- **Train of thought (reasoning)**

The following prompt idea goes in a similar direction. Instead of re-

requesting *one* specific solution, you can ask the AI chatbot to put itself in the role of several experts discussing a question like the following example:

Imagine three IT experts working on a question. They discuss various proposals, discard unsuitable ideas, and finally agree on a solution. The task is as follows: How can a redundant backup system be designed for a web server?

- **Working step by step**

For complex tasks, you should avoid trying to fit everything into one prompt at once. It works much better if you proceed step-by-step and first ask the AI chatbot for an outline and then take care of the details in further prompts. This procedure is also known as *prompt chaining*. The first prompt may look as follows:

I need to design a database to analyze our web traffic. First of all, please give me an outline of the database. (What tables do I need?) We'll discuss the details of each table later.

Note that ChatGPT and similar services have strict limits for the maximum input and output length (size of the context window and maximum number of output tokens). AI chatbots will therefore never provide answers that consist of several thousand lines of code. It's also not advisable to pass a file with many lines of code to the prompt and request a complete revision (e.g., refactoring). Instead, you need to break your task down into parts and carry out each step separately.

- **New session on change of topic**

Start a new session when you change the topic. ChatGPT takes the information collected in a session into account for follow-up questions. All previous questions and answers therefore form the context for all further questions.

However, if a new question relates to a different programming language, a different problem or a different library, it's advisable to start again in a separate session.

A restart can also be useful if you're dissatisfied with the results so far or if you think that you or the AI tool have reached a dead end. The context from the preceding questions and answers may prevent ChatGPT from pursuing other ideas.

- **Understanding and testing code**

The AI tool isn't responsible for your code—you are! Don't just hope

that the generated code will work; instead, test it thoroughly, taking all conceivable special cases into account. If you can't understand the AI-generated code, you need to familiarize yourself with the underlying concepts, even if it's inconvenient at the moment.

- **Take particular care with new or exotic topics**

As a matter of principle, language models have difficulty with new technologies or peripheral IT areas. On the one hand, there is little training material available; on the other hand, the problem is exacerbated by the fact that the training for a current language model started usually at least 6 months and often 12 or more months previously.

Colloquial Prompts Also Work, But Not Always Perfectly

In this book, we've endeavored to formulate the prompts in neat, complete sentences. The entire text of this book has been proofread, and prompts in poor English simply don't fit in.

But don't worry: AI tools usually understand you even if you formulate the prompt very briefly. Prompts such as the aforementioned *Do better!*, *Optimize this code!*, *How does this work?*, or *Why doesn't this work?* are also fine.

However, it's important that the intention of your question is unambiguously clear. A precise formulation helps both you and the AI tool better understand the problem. In this respect, the time spent on a "proper" formulation of a prompt is usually well invested.

1.2.1 Prompt Frameworks

Compliance with the rules of various prompt frameworks can help you with prompting. In this context, a "framework" is a kind of guide to the components from which a prompt can be assembled.

The most popular framework is the RTF framework. In the prompt, you formulate the *role* of the AI tool, the specific *task*, and the desired *format* of the response. In the following example, we've labeled the components of the prompt. In real life, of course, these markings are omitted.

Prompt: (*Role*) You're an experienced Python developer. (*Task*) Sketch the framework of a REST API for a to-do app using Django. (*Format*) Present the answer in a step-by-step manner. Just show the outline of the code; don't go into details yet.

Alternatives to RTF are the RODES framework (role, objective, details, example, sense check) and the RISEN framework (role, instructions, steps, end goal, narrowing). However, considerable effort is required in formulating that many details. In this respect, the use of such frameworks is more suitable if you program an AI chatbot or another application yourself using an API (see also [Chapter 10](#)).

The use of the frameworks outlined here for the prompt structure is optional. Even if you only specify the task (i.e., the *task* in the sense of the RTF framework), you'll achieve useful results. But especially in tricky cases, adhering to framework rules can lead to better results.

1.2.2 A Question of Context

We've already pointed out that AI tools require additional information, that is, context, to provide relevant answers. The context determines which data is actively processed outside the actual language model. In a ChatGPT session, the context results from all questions (prompts) asked in a session so far and the resulting answers.

The size of the context of language models is limited. It's normally measured in tokens (see [Table 1.1](#)). More technical details about context and tokens will follow in [Section 1.5](#). Some language models have further limits. The output of ChatGPT-4o is limited to 4,000 tokens. This means that ChatGPT-4o can only deliver approximately 12 KB of code at a time despite a comparatively large context memory. The output limit of 4,000 tokens also applies to the *Claude 3 Opus* language model from Anthropic. It was increased to 8,000 tokens for *Claude 3.5 Sonnet*.

Language Model	Max. Context Size	Max. Text Quantity	Max. Code Quantity
GPT-3.5 Turbo	4,096 tokens	~16,000 characters	~12 KB
GPT-4	8,000 tokens	~32,000 characters	~24 KB
GPT-4o (omni)	32,000 tokens	~128,000 characters	~96 KB
Llama 3.3	128,000 tokens	~500,000 characters	~380 KB
Claude 3 Opus	200,000 tokens	~800,000 characters	~600 KB
Claude 3.5 Sonnet	200,000 tokens	~800,000 characters	~600 KB

Table 1.1 Maximum Context Size of Various Language Models from OpenAI, Meta, and Anthropic (as of January 2025)

In the future, even the considerable context size of the Anthropic models could look modest. In August 2024, the Magic AI company presented a language model that can cope with a context window of 100 million tokens. This corresponds to approximately 10 million lines of code. For comparison, the entire kernel of Linux, which is one of the largest public software projects, comprises around 30 million lines of code.

What do the maximum context size (often also the *context window*) and the maximum output have to do with the prompt? We’ve had the best experiences with the use of AI tools when we’ve limited ourselves to small, manageable functionalities. Don’t ask for everything at once (“Write code for

hotel booking software like booking.com.”); instead, first design the structure of your program, and then apply AI tools to its individual functions.

GPTs Are Stateless

You’re used to ChatGPT and other tools allowing you to refer to the questions and answers from the previous chat history in follow-up questions. This gives you the impression that ChatGPT is memorizing the conversation.

However, language models actually work *statelessly*; that is, they don’t remember anything and regard every prompt as completely new. The impression of memory is only created by the programming of the chat interface. From the second prompt, the website not only transmits the new prompt to the voice system but also all previous questions and answers. These form the context for the new question and therefore also the “memory” for the chat history.

In [Chapter 10](#), we describe how the programming of a minimal chat system works behind the scenes.

1.2.3 Custom GPTs

If you repeatedly specify certain rules, it may be worth setting up a customized *generative pretrained transformer* (GPT). (We’ll explain the term GPT fully in [Section 1.5](#).) With ChatGPT, defining your own GPTs requires a paid version of ChatGPT. In the ChatGPT web interface, first click on **Explore GPTs** and then on **Create**. In the GPT editor, go to the **Configure**

dialog box. There, you give your own GPT a name, briefly describe the intended use, and then specify the instructions for the GPT.

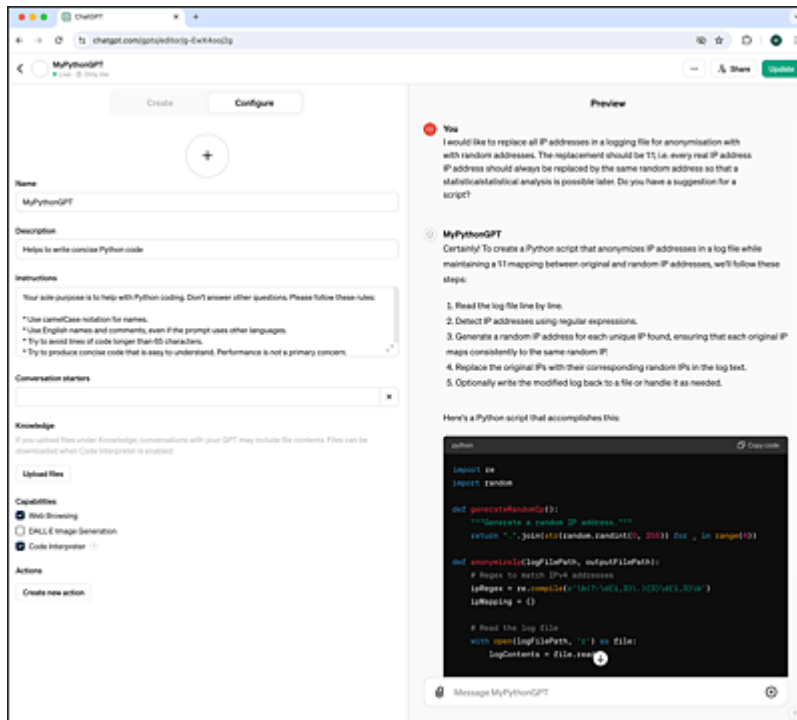


Figure 1.2 Instructions for the New GPT on the Left, and an Application Example on the Right

These instructions can look as follows, for example:

Instructions: Your sole purpose is to help with Python coding. Don't answer other questions. Follow these rules:

- Use English names and comments, even if the prompt uses other languages.
- Use camelCase notation for names.
- Try to avoid lines of code longer than 65 characters.
- Try to produce concise code that is easy to understand. Performance isn't a primary concern.
- If possible, try to avoid modules that need to be installed externally (pip).

Depending on the application, you can use the **Upload files** button to upload files with additional information (e.g., the documentation for a new software library or version that wasn't yet available when the language model was trained), which we haven't done in this example. Permitted doc-

ument formats are plain text, Markdown, HTML, and PDF. You can try out your individual GPT right away in the **Preview** area.

In [Figure 1.2](#), you can see that the new coding wizard works as intended:

- The prompt doesn't contain any information about which programming language ChatGPT should use. But due to the individual GPT instructions, ChatGPT knows that it should use Python.
- Function and variable names use camelCase notation.

The **Code Interpreter** checkbox allows the GPT to execute code directly, for example, to perform a test run. However, this only works for a few programming languages and is also associated with various restrictions, such as terms of runtime or access to files or the internet.

Prompt: Can you show me a script to calculate Fibonacci numbers? Please run the script and calculate the 25th Fibonacci number.

```
def fibonacci(n):  
    """Calculate the n-th Fibonacci number."""  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a  
  
# Calculate the 25th Fibonacci number  
fibonacci(25)
```

The 25th Fibonacci number is 75,025. If you need to calculate another Fibonacci number, you can use the `fibonacci` function provided in the script by changing the input value.

1.2.4 Comparing Custom GPTs, Fine-Tuning, and Retrieval-Augmented Generation

Custom GPTs primarily change the role of the language model, that is, its basic task. Usually, the role description is “You are a helpful wizard” or something similar. We’ll go into the concept of role, prompt, and response

in more detail in [Chapter 10](#). A changed role influences the answers of the language model, but doesn't change its core.

You have even more influence on the functionality of a given language model via *fine-tuning*. In the simplest case, you give the language model a list of questions (prompts) and what you consider to be the best answers. The language model can be based on this. However, ChatGPT doesn't currently support fine-tuning for custom GPTs.

The “real” integration of additional knowledge that wasn't yet available when the language model was trained is provided by *retrieval-augmented generation* (RAG; see also [Chapter 12](#)). This procedure is also used for custom GPTs if you upload additional knowledge files.

Unfortunately, it's impossible to simply retrain a language model for a specific application or to take additional information into account. This approach consumes an extremely large amount of computing time and money and is therefore only feasible for large (AI) companies.

The Limits of Custom GPTs

Integrating a few PDF files with additional information and changing the system instruction (role) of the language model isn't enough to build a well-functioning company-specific or application-specific bot in just a few minutes. Such attempts have always failed miserably in the past.